

Verification Recommendations for the Repaired Split-Forest Decidability Proof

Prepared for independent adversarial review

May 12, 2026

Abstract

This memorandum gives a concrete verification plan for the repaired decidability proof for $\mathcal{ALCT}_{\text{RCC5}}$. It is written as instructions for an independent reviewer or LLM assistant, for example Claude, and is intended to turn the remaining proof risks into checkable work packages. The recommended review stack is: adversarial proof reading, executable finite counterexample search, and selective Lean formalization of the core lemmas. The aim is not merely to endorse the proof, but to try to falsify it in the places where errors are most likely: RCC5 composition, witness thinning, typed equality quotienting, blocking versus semantic equality, multiple upward proper-part witnesses, request-closed cyclic blocking, and support-closed sibling mosaics.

1 Purpose and scope

The current repaired proof uses the following architecture.

- (1) First, a witness-generated submodel lemma reduces arbitrary abstract RCC5/Kripke models to sparse models generated by selected existential witnesses in the finite closure $\text{Cl}(C_0)$.
- (2) Second, the thinned model is represented by a containment-oriented generated split forest. Vertical generated cover edges present PP/PPI containment; they are not assumed to be Hasse edges of an arbitrary original semantic model.
- (3) Third, split copies handle multiple selected containment parents and are later synchronized by an explicit typed EQ-congruence.
- (4) Fourth, non-vertical DR/PO interactions and side witnesses are stored in support-closed sibling mosaics.
- (5) Fifth, infinite branches are represented by finite regular certificates with request-closed cycles. Blocking repetition is a finite presentation device and must not be confused with semantic equality.

This document recommends a verification program for that architecture. It deliberately does not ask the reviewer to replace the proof by a different automata-theoretic proof. Automata terminology may be used internally if useful, but the final proof target should remain the original split-forest architecture with direct finite request-cycle checks.

2 Executive recommendation

Use a three-layer review.

Layer	Role
Adversarial proof review	Read the manuscript line by line and attempt to construct countermodels or invalid certificates. Pay special attention to equality, splitting, and mosaic synchronization.
Computational analysis	Implement finite counterexample searches for RCC5 composition, certificate soundness, small-model/certificate comparison, multiple-superpart stress tests, request-cycle tests, and mosaic closure tests.
Lean-assisted formalization	Formalize the small but central lemmas: closure and NNF complement, witness thinning, typed-EQ quotient soundness, blocking-as-fresh-unfolding, request-closed lasso correctness, and mosaic composition lifting.

LLM agreement is useful but not decisive. Claude should be used as an independent adversarial reviewer, but the strongest confidence increase will come from executable tests and formalized core lemmas.

3 Critical proof obligations to audit

The independent reviewer should explicitly check the following obligations.

Check 1 (RCC5 relation algebra). The manuscript must use the correct RCC5 composition relation. In particular,

$$PP \circ PP \subseteq \{PP\}, \quad PPI \circ PPI \subseteq \{PPI\},$$

while compositions involving DR are generally not singleton. The table should be validated by finite set semantics over nonempty subsets of small universes.

Check 2 (Closure and duals). The finite closure $Cl(C_0)$ must be closed under NNF complement $D \mapsto \overline{D}$, including

$$\overline{\exists R.D} = \forall R.\overline{D}, \quad \overline{\forall R.D} = \exists R.\overline{D}.$$

This is necessary for the witness-generated submodel lemma, because failed universals are witnessed by existential duals.

Check 3 (Witness thinning). If $\mathcal{I}, a_0 \models C_0$, the selected-witness substructure $\mathcal{I} \upharpoonright W$ must still satisfy all formulas in $Cl(C_0)$ at all generated points. Existentials are preserved because their witnesses are kept; universals are preserved because restriction deletes successors. The proof should be formalized.

Check 4 (Generated cover, not arbitrary Hasse extraction). The proof must not require tree edges to be Hasse-cover edges of an arbitrary starting model. Tree edges are selected generated containment-cover edges in the constructed presentation. Semantic PP/PPI is then interpreted by strict ancestor/descendant closure plus the finite non-vertical interface labels.

Check 5 (Blocking is not semantic equality). The proof must keep separate

$$\equiv_{\text{blk}} \quad \text{finite blocking/profile repetition}$$

from

$\equiv_{\text{EQ}}$ semantic equality or weak-EQ mate relation.

A cycle in the finite certificate unfolds to fresh semantic occurrences. Reusing a profile in the next lap must not imply EQ.

Check 6 (Multiple upward PP witnesses). Because the containment orientation has parent equal to proper superpart, $\exists \text{PP}.D$ requirements point upward. A single occurrence may have only one parent chain, but a semantic element may need several incomparable superpart witnesses. The proof must discharge such requirements through explicit EQ-mate occurrences and prove that quotienting makes the witnesses simultaneous for the collapsed element.

Check 7 (Typed EQ quotient). The typed EQ-congruence must be explicit. If $x \equiv_{\text{EQ}} y$, then types, role obligations, external relation labels, side interfaces, and witness behavior must agree sufficiently for quotienting to preserve concept truth and RCC5 consistency. No profile equality or blocking equality may automatically generate $\equiv_{\text{EQ}}$.

Check 8 (Support-closed mosaics). Mosaics must record joint realizability, not merely pair tables. The proof must make explicit how every triple in the unfolding is covered by a compatible 3-mosaic, and how larger finite prefixes are obtained by overlap-compatible amalgamation. If a finite arity bound is claimed, it must be stated and justified.

Check 9 (Request-closed cycles). For transitive PP/PPI roles, finite cyclic blocking is sound only if each existential request is fulfilled along the finite stem/cycle or is discharged by an explicit equality-mate route that unfolds to a later fresh occurrence. Individual reachability is not enough if the witnesses cannot be realized simultaneously in one unfolding.

4 Computational verification work packages

Computational checks should be designed as counterexample engines. A successful run does not prove the theorem, but a failing run gives a concrete repair target.

4.1 WP1: RCC5 composition-table checker

Task 1. Implement a program that computes RCC5 base relations on nonempty subsets of a finite universe and derives the composition table by exhaustive enumeration.

Definitions for a finite universe U and nonempty subsets $A, B \subseteq U$:

$$\begin{aligned} \text{AEQB} &\iff A = B, \\ \text{APPB} &\iff A \subsetneq B, & \text{APPIB} &\iff B \subsetneq A, \\ \text{ADRB} &\iff A \cap B = \emptyset, \\ \text{APOB} &\iff A \cap B \neq \emptyset, & A &\not\subseteq B, & B &\not\subseteq A. \end{aligned}$$

For each pair (R, S) , compute all T for which there exist nonempty A, B, C with ARB , BSC , and ATC . Increase $|U|$ until the table stabilizes; $|U| = 4$ or $|U| = 5$ is typically enough for RCC5 base relations, but the script should report stabilization data.

Deliverable 1. A script `rcc5_composition_check.py`.

A generated table `rcc5_composition_table.txt`.

The script should fail if the manuscript table differs from the computed table.

4.2 WP2: bounded certificate soundness fuzzer

Task 2. Implement a bounded generator for finite split-forest certificates and a finite-depth unfolding checker. Search for certificates accepted by the proposed validity rules whose unfoldings violate JEPD, RCC5 composition, typed EQ, or concept satisfaction.

The fuzzer should include independent toggles for the delicate mechanisms:

- (a) blocking cycles versus explicit EQ-mates;
- (b) one-parent containment chains versus multiple upward PP witnesses;
- (c) side DR/PO witnesses;
- (d) mosaic closure depth;
- (e) request-closed cycle checks.

Deliverable 2. A script `certificate_fuzzer.py`. If it finds a failure, it should output a minimized certificate and a finite prefix witnessing the violation.

4.3 WP3: small-model comparison oracle

Task 3. For small closures, compare brute-force finite abstract RCC5/Kripke models against the certificate checker.

Generate small concept sets over a few concept names and roles $\{EQ, PP, PPI, PO, DR\}$. For domains of size $n \leq 4$ or 5, enumerate JEPD RCC5 labelings satisfying composition. Then test satisfiability of concepts directly and compare with the certificate procedure restricted to a comparable bound.

Deliverable 3. A script `small_model_oracle.py` and a report listing all mismatches. Mismatches should be classified as possible incompleteness, possible unsoundness, or bound artefact.

4.4 WP4: multiple-superpart stress tests

Task 4. Generate and test concepts that force several incomparable PP-successors/superparts of the same semantic element.

A basic stress schema is

$$\exists PP.A \sqcap \exists PP.B \sqcap \forall PP(A \rightarrow (\forall PP.\neg B \sqcap \forall PPI.\neg B)) \sqcap \forall PP(B \rightarrow (\forall PP.\neg A \sqcap \forall PPI.\neg A)).$$

The intended satisfying pattern has one element with two incomparable superparts, one satisfying A and one satisfying B . The checker should verify that the split-copy and EQ-aware reachability machinery accepts this pattern without forcing the two witnesses onto a single parent chain.

Deliverable 4. A stress-suite file `multiple_superpart_tests.json` and a short report explaining whether each schema is accepted or rejected and why.

4.5 WP5: blocking-cycle request tests

Task 5. Test cyclic profiles carrying transitive existential requests.

Positive test:

$$\exists PP.T \sqcap \forall PP.\exists PP.T$$

should be accepted by a one-state upward cycle whose repeated occurrences are fresh.

Negative test: a cycle that carries $\exists PP.A$ but has no later occurrence satisfying A should be rejected.

Deliverable 5. A script `request_cycle_tests.py` showing accepted and rejected cyclic profiles, with unfolded finite-prefix witnesses.

4.6 WP6: mosaic closure counterexample search

Task 6. Search for locally accepted mosaics whose pair labels cannot be globally synchronized without violating RCC5 composition.

The search should construct finite sets of abstract occurrences with pair labels and type labels. It should then check whether the support-closure rules guarantee that every triple (x, y, z) satisfies

$$\ell(x, z) \in \ell(x, y) \circ \ell(y, z).$$

It should also test whether side witnesses for DR/PO can be inserted without breaking existing universal restrictions.

Deliverable 6. A script `mosaic_closure_search.py`. Any counterexample should include the minimal set of profiles, pair labels, and missing support condition.

5 Lean-assisted formalization work packages

The Lean effort should be incremental. Do not start by formalizing the whole proof. Formalize the small core lemmas that are most likely to hide mistakes.

5.1 Lean target L1: RCC5 relation algebra

Task 7. Define the finite type of RCC5 base relations and the computed composition function. Prove the basic algebraic facts used by the paper.

Suggested theorem statements:

$$\begin{aligned} \forall R, \quad EQ \circ R &= \{R\} = R \circ EQ, \\ PP \circ PP &= \{PP\}, \\ PPI \circ PPI &= \{PPI\}, \\ \text{conv}(R \circ S) &= \text{conv}(S) \circ \text{conv}(R). \end{aligned}$$

Deliverable 7. `ALCIRCC5/RCC5RelAlg.lean`.

5.2 Lean target L2: closure, NNF complement, and Hintikka types

Task 8. Formalize concepts, NNF complement, finite closure, and local Hintikka type conditions.

The critical property is that for every $D \in \text{Cl}(C_0)$, the complement $\overline{D} \in \text{Cl}(C_0)$, and satisfaction obeys

$$\mathcal{I}, x \models \overline{D} \iff \mathcal{I}, x \not\models D.$$

Deliverable 8. `ALCIRCC5/SyntaxClosure.lean`.

5.3 Lean target L3: witness-generated submodel lemma

Task 9. Formalize the selected-witness construction and prove preservation of closure formulas under restriction to the generated subdomain.

Target theorem:

$$\mathcal{I}, a_0 \models C_0 \implies \mathcal{I} \upharpoonright W, a_0 \models C_0,$$

where W is the least set containing a_0 and closed under selected witnesses for all true existential formulas $\exists R.D \in \text{Cl}(C_0)$.

More generally, prove:

$$\forall x \in W, \forall D \in \text{Cl}(C_0), \quad \mathcal{I}, x \models D \iff \mathcal{I} \upharpoonright W, x \models D.$$

Deliverable 9. `ALCIRCC5/WitnessThinning.lean`.

5.4 Lean target L4: typed EQ quotient soundness

Task 10. Formalize weak split structures with an explicit equivalence relation $\equiv_{\text{EQ}}$. Assume $\equiv_{\text{EQ}}$ is a typed RCC5 congruence and prove that quotienting preserves RCC5 frame conditions and truth of closure formulas.

The equivalence relation $\equiv_{\text{EQ}}$ must not be generated by profile equality or by blocking repetition. It is a separate semantic equality relation.

Target theorem: If $\mathcal{S}$ is a weak split structure satisfying typed-EQ congruence and local truth conditions, then the quotient $\mathcal{S}/\equiv_{\text{EQ}}$ is a strong-EQ RCC5 model and preserves all closure formulas.

Deliverable 10. `ALCIRCC5/EQQuotient.lean`.

5.5 Lean target L5: blocking unfolds to fresh occurrences

Task 11. Formalize finite cyclic certificates and their infinite unfolding. Prove that repeated profiles are fresh occurrences unless explicitly identified by $\equiv_{\text{EQ}}$.

Test theorem: The one-state cycle with a proper-superpart edge unfolds to an infinite strict chain

$$u_0 \text{PP} u_1 \text{PP} u_2 \text{PP} \dots$$

with no $u_i \text{EQ} u_j$ for $i \neq j$, unless an explicit $\equiv_{\text{EQ}}$ declaration says otherwise. In particular, no PP-cycle is created by blocking.

Deliverable 11. `ALCIRCC5/BlockingUnfolding.lean`.

5.6 Lean target L6: request-closed lasso correctness

Task 12. Formalize the finite request-closed cycle condition and prove that the infinite unfolding satisfies all transitive PP/PPI existential and universal obligations.

For a cycle $v_0, \dots, v_{k-1}$, the finite condition should imply:

$$\exists \text{PP}. D \in \text{tp}(v_i) \Rightarrow \exists j > i \text{ in the infinite unfolding with } D \in \text{tp}(v_j),$$

with the analogous condition for PPI. Universal obligations should propagate by induction along strict ancestor/descendant closure.

Deliverable 12. `ALCIRCC5/LassoRequests.lean`.

5.7 Lean target L7: mosaic composition lifting

Task 13. After the computational mosaic rules stabilize, formalize the local-to-global composition lifting lemma.

Target theorem: If every pair label in an unfolded finite prefix is assigned by a compatible support-closed mosaic system, and every triple is covered by an agreeing 3-mosaic, then global RCC5 composition holds for that finite prefix.

Deliverable 13. `ALCIRCC5/MosaicComposition.lean`. This target may be postponed until the Python counterexample search no longer finds defects in the mosaic rules.

6 Suggested order of work for Claude

Claude should carry out the work in the following sequence.

- Phase 1: **Adversarial read-through.** Produce a line-by-line list of all definitions that mention equality, profile repetition, parent contexts, sibling mosaics, or request closure. Identify any place where $\equiv_{\text{blk}}$ and $\equiv_{\text{EQ}}$ might be conflated.
- Phase 2: **RCC5 table validation.** Implement WP1 first. No later verification is meaningful if the composition table is wrong.
- Phase 3: **Syntax and thinning.** Formalize or at least mechanize L2 and L3. This certifies that the proof may restrict attention to witness-generated models.
- Phase 4: **Blocking and equality.** Implement WP5 and Lean targets L4-L6. These address the highest-risk part of the proof.
- Phase 5: **Stress tests.** Implement WP2-WP4. Search especially for invalid certificates involving multiple incomparable superpart witnesses.
- Phase 6: **Mosaics.** Implement WP6. Only after the computational rules stabilize, attempt L7.
- Phase 7: **Final audit.** Return a table of proof obligations with status: proved, tested only, counterexample found, or still open.

7 Expected final report format

The final response from Claude or another reviewer should have the following structure.

- (1) **Summary verdict.** State whether the current proof is accepted, rejected, or conditionally accepted pending specified repairs.
- (2) **Proof-obligation table.** For each obligation in Section 3, report status and evidence.
- (3) **Executable artifacts.** List all Python scripts, command lines, and outputs.
- (4) **Lean artifacts.** List all Lean files, theorem names, and any axioms or admitted proofs. Any use of `sorry` must be explicitly reported.
- (5) **Counterexamples.** If any are found, provide the smallest certificate/concept/frame and explain the violated proof claim.
- (6) **Remaining gaps.** State exactly which lemmas are still informal.
- (7) **Recommended manuscript edits.** Give concrete replacement text or theorem statements for the proof.

8 Pass/fail criteria

The proof should not be called fully established unless the following minimum criteria are met.

- (C1) The RCC5 composition table is executable and independently validated.
- (C2) The witness-generated submodel lemma is formalized or proved in a level of detail equivalent to formalization.
- (C3) Blocking repetition is formally separated from semantic EQ.
- (C4) Typed-EQ quotient soundness is proved as a standalone lemma.
- (C5) Multiple upward PP-witness examples are accepted by the certificate rules and sound after quotienting.
- (C6) Request-closed cycles are proved to satisfy all transitive eventualities in the infinite unfolding.
- (C7) Mosaics have an explicit finite arity bound or an explicit finite closure construction sufficient for every finite prefix used in the soundness proof.
- (C8) The decision procedure enumerates a finite, effectively checkable certificate language.

If criteria C1-C6 are met but C7 remains informal, the result should be described as a strong proof strategy with one remaining finite-interface gap. If C7 is also closed, then the proof is much closer to a publication-grade decidability theorem.

9 Remarks on avoiding explicit automata

The repaired proof may avoid explicit Buechi or parity automata terminology. However, it cannot avoid the mathematical content of fairness. The finite certificate must still include a request-closed cycle condition. The correct non-automata presentation is:

A blocked cycle is valid only if every transitive PP/PPI existential request occurring on the cycle is fulfilled somewhere later on the same finite cycle, or through an explicitly recorded equality-mate route whose unfolded occurrences are fresh and later in the relevant direction.

This is equivalent in spirit to an acceptance condition, but it can be stated directly as a finite graph reachability and cycle-closure condition.

10 Bottom line

The recommended verification campaign is designed to answer one question: is the repaired split-forest certificate language both sound and complete for satisfiability of $\mathcal{ALCI}_{\text{RCC5}}$? The most valuable immediate work is not another prose rewrite. It is to formalize and test the specific mechanisms that remain delicate:

RCC5 composition, witness thinning, and typed equality quotienting,
blocking-as-fresh-unfolding, request-closed cycles,
and support-closed mosaics.

If those mechanisms withstand adversarial review, finite counterexample search, and selective Lean formalization, confidence in the decidability proof will increase substantially.

A Minimal prompt for an independent LLM reviewer

The following prompt can be given to Claude together with the repaired proof manuscript and this memorandum.

You are reviewing a proposed decidability proof for $\mathcal{ALCI}_{\text{RCC5}}$. The proof uses witness thinning, generated containment split forests, split copies for multiple containment parents, support-closed sibling mosaics, typed equality quotienting, and finite request-closed blocking cycles. Please do not merely summarize the proof. Try to falsify it. Focus on: RCC5 composition, closure under NNF complement, the witness-generated submodel lemma, separation of blocking from semantic equality, typed-EQ quotienting, multiple incomparable PP-superpart witnesses, request-closed cyclic blocking, and finite support-closed mosaic arity. Produce: (i) a proof-obligation table, (ii) any counterexamples or suspect schemas, (iii) executable test suggestions or code, (iv) Lean theorem statements that would certify the delicate lemmas, and (v) concrete manuscript edits.

B Suggested directory layout

```
verification-pack/  
  README.md  
  python/  
    rcc5_composition_check.py
```

```
certificate_fuzzer.py
small_model_oracle.py
multiple_superpart_tests.json
request_cycle_tests.py
mosaic_closure_search.py
lean/
  ALCIRCC5/
    RCC5RelAlg.lean
    SyntaxClosure.lean
    WitnessThinning.lean
    EQQuotient.lean
    BlockingUnfolding.lean
    LassoRequests.lean
    MosaicComposition.lean
  reports/
    rcc5_composition_table.txt
    fuzzer_findings.md
    small_model_mismatches.md
    lean_status.md
    final_review.md
```